CRYSTAL
PRACTICE MANAGEMENT

CRYSTAL PM

HOW-TO GUIDE

Connect Crystal Meet rooms to Zoom

Let each room join Zoom meetings through Zoom's Meeting SDK, including meetings hosted by customers and vendors, with no one signing in on the device.

This guide has two halves. The Zoom half is done once for all rooms by the person who administers your Zoom account: two apps in the App Marketplace, one review request, and one Zoom user per room, about 30 minutes at the keyboard. Zoom's review of the app then runs for a few weeks in the background; rooms join meetings on your own account in the meantime. The device half takes about five minutes per room. Rooms still join only when someone presses Join on the room page.

STEP 1
The SDK app
A Meeting SDK app and its client id and secret.

STEP 2
Zoom's review
Submit the app, unlisted, so rooms may join outside meetings.

STEP 3
The token app
A Server-to-Server OAuth app with two read-only scopes.

STEP 4
A user per room
Rooms join outside meetings as their own Zoom user.

STEP 5
On the device
The credentials file, the installer, the check.

STEP 6
A test meeting
One inside your account, one outside.

BEFORE YOU BEGIN

Access. The Zoom account owner, or an admin whose role has the **Zoom for developers** permission (then **Develop** shows at the top of the App Marketplace) and who may add users. The owner grants that permission at `admin.zoom.us` under **User Management › Roles**: edit the admin's role, then **Role Settings › Advanced features › Zoom for developers**, tick **View** and **Edit**.

Why four parts. Zoom lets an SDK app join meetings hosted by other Zoom accounts only once Zoom has reviewed the app, and, since March 2026, only as a signed-in Zoom user. The SDK app (step 1) joins your own meetings on its own; the review (step 2), the token app (step 3) and a Zoom user per room (step 4) let the room join everyone else's.

Secrets. You will collect a client id and secret (step 1) and an account id, client id and secret (step 3). Keep them out of email and chat; they go straight into a file on each device.

1 Create the Meeting SDK app

- **Sign in to the App Marketplace** at `marketplace.zoom.us` with the account owner's login, or as an admin with the permission above. Open **Develop** at the top right and choose **Build App**. If there is no Develop menu, the permission is missing; see Before you begin.
- **Pick the type.** In the dialog choose **General App** and press **Create**. Zoom folded its old "Meeting SDK app" type into the General App; there is no separate SDK type to look for.
- **Name it and make it account-owned.** On **Basic Information** set the name to `Crystal Meet rooms` and choose **Admin-managed** where the page asks how the app is managed. It belongs to the account; nobody signs in to it.
- **Copy the development credentials.** Still on Basic Information, **App Credentials** shows two pairs, **Development** and **Production**. Copy the Development **Client ID** and **Client Secret**; they go into the credentials file as **sdk_client_id** and **sdk_client_secret**. The development pair joins meetings on your own account straight away. The Production pair is for after step 2.
- **If the page insists on an OAuth Redirect URL** before it lets you continue, enter `https://crystalpm.com/`. The rooms never use it; it only has to be a page on a domain you own.
- **Turn on the Meeting SDK.** Open **Features** and its **Embed** tab (some layouts show Embed under **Surface**), switch **Meeting SDK** on and save. Leave the three switches beneath it off: **programmatic join use case** (the room joins when a person presses Join, as a Zoom user, so no exception is needed), **Request Anonymous Join Exception** (that is for apps that cannot use a ZAK; the rooms use one) and **Use Device OAuth** (the token app in step 3 does that job). The SDK downloads listed there are not needed: the room loads the web SDK itself.
- **Nothing else on the other pages.** **Access** (secret token, event subscriptions, Plugin SDK), **Surface** (home URL, domain allow list, products, in-client features), **Connect** and **Actions and Triggers** all stay as they are; the room never shows anything inside the Zoom client. On **Local Test**, **Add App Now** is harmless either way.
- **Add the scope Zoom insists on.** The build flow will not continue with an empty **Scopes** page. Press **Add Scopes**, search for `zak` and add `user:read:zak:admin` (View a user's ZAK). If it is not offered, search for `token` and add `user:read:token:admin` instead; if the list only has older names, pick **User › View all user information**, the one Zoom's own SDK guide names for admin-managed apps. In **Scope Description** write: "Fetches a conference-room user's ZAK so the room device can join meetings hosted by other Zoom accounts as that user." Add nothing else; every extra scope lengthens step 2.
- **Skip Local Test.** The rooms do not sign in through the app, so it needs no event subscriptions and no "Add App Now" to work.

  You should now see the app under Manage › Built Apps (Zoom also calls this list Created apps) with Meeting SDK on, and have its development client id and secret.

2 Submit the app to Zoom's review, unlisted

Needed only for meetings hosted by other Zoom accounts, which is most customer and vendor meetings. Zoom's rule: an SDK app may join meetings outside the account that created it only after

Zoom has reviewed the app. Reviewed does not mean public; the app can be unlisted afterwards. Zoom asks developers to submit at least a month ahead, so do this the same day as step 1.

- **Ask to publish first.** A new app is private and shows "Ready for local test" with no way to submit. In the Marketplace open **Developer › Created apps**, press the **⋯** menu at the end of the app's row and choose **Request to publish** (older layouts: **Manage**, the app, **Activation**, **Share your app with others › Change Now**, then **OK** in the **Enable Publishing** window). The left menu gains **App Listing**, **Technical Design Document**, **Beta Test** and **Publish**, and the **Production** switch at the top shows the fields they need.
- **Fill in the listing** under **Production › App Listing**. The Publish page lists what is still missing, in five groups: Basic Information, App Information, Links & Support, EU & Discoverability (where **Set my app as 'Unlisted'** lives) and Technical Design. Ready-made text for every field, plus an icon and a gallery image, are in `deploy/zoom-marketplace/` in the repository. In short: a 160 by 160 icon, a short and a long description, for example "Crystal PM's conference-room devices join Zoom meetings from the room's TV. Used only inside Crystal PM.", one image for the gallery, the company name, a developer contact mailbox someone reads, and a category. Where it asks for a support address and privacy policy, use pages on `crystalpm.com`.
- **Write the Technical Design Document.** Zoom will not accept a submission without it. A page is enough: what the app is (a Raspberry Pi behind a conference-room TV that joins meetings as a participant when a person presses Join), what data it touches (a ZAK for the room's own Zoom user, nothing stored beyond the device), and that there are no outside users.
- **The EU fields.** **App Listing › EU & Discoverability** asks for business details down to a bank account's last digits and an identification document. Those are the EU Digital Services Act's "trader" details for apps offered to EU users; an internal device app is not a trader. Email `integration.testers@zoom.us` saying so and asking for the fields to be removed from the build flow, which Zoom's documentation offers, or turn off **List my app in the EU** in that section. The exact wording is in `deploy/zoom-marketplace/listing.md`.
- **Verify the domain when asked.** Every web address in the listing must be on a domain Zoom has verified for your account; the **Publish** page offers a DNS record, a file on the site or a meta tag. Whoever manages crystalpm.com can do it in minutes.
- **On the Publish page** pick the audience **External Zoom users**: that is the choice that goes through Zoom's review, which is what lets the rooms join other accounts' meetings, including ones a customer invites the room to. It does not hand the app to anyone: it stays unlisted after the approval, and it has no screens or sign-in of its own, so it is useless without the credentials on your devices. **Users within my Zoom account** skips the review and would leave the rooms limited to meetings hosted by your own account. In the release notes for the reviewer say the app runs on a Raspberry Pi behind a conference-room TV, joins as an ordinary participant when a person presses Join, has no sign-in flow and no outside users, and is device-specific; offer a short screen recording of a join, or a live session with the review team. Write "none" where it asks for test logins, choose whether Zoom activates the app on approval, accept the Marketplace Development Agreement and submit.

- **After the approval,** on **Manage › Created Apps** use the app's menu to **Set my app as 'Unlisted'** so it never appears in the public Marketplace, then copy the **Production** Client ID and Client Secret from App Credentials into each room's credentials file in place of the development pair (step 5) and restart the service. Zoom's reviewers approve the production pair; the development pair stays limited to your own account's meetings. Zoom's reviewers reply by email with questions; answer them quickly to keep the clock moving.

  You should now see the app's status as In review, and later Approved. Until then rooms join meetings hosted on your own account only.

3 Create the Server-to-Server app for tokens

- **Build it.** **Develop › Build App**, choose **Server-to-Server OAuth** and press **Create**; name it `Crystal Meet room tokens`.
- **Copy the credentials** from **App Credentials**: **Account ID**, **Client ID** and **Client Secret**.
- **Fill in Information.** Zoom will not activate the app without a short description, the company name and a developer contact name and email. Leave event subscriptions off on the **Feature** page.
- **Add two read-only scopes.** Under **Scopes › Add Scopes**, search for `zak` and add `user:read:zak:admin` (View a user's ZAK), then search for `token` and add `user:read:token:admin` (View a user's token). Zoom's docs name the first for ZAKs and the second for the token request the room makes, so add both; nothing else is needed. The list only offers scopes the signed-in person's own role allows, so if either is missing, do this as the account owner.
- **Activate the app** on the **Activation** page. Tokens only work while it is active.

  You should now see the app listed as active, with the account id, client id and secret saved with the step 1 values.

4 Create a Zoom user for each room

- **Add the user.** In `admin.zoom.us` go to **User Management › Users › Add Users**. Use a mailbox you can read, for example `room1@crystalpm.com`, and the **Basic** user type, which is free on a paid account.
- **Activate it.** Open the invitation email in that mailbox and finish the sign-up.
- **Name it after the room.** In the user's profile set the display name to the room's name, for example `Room 1`. That is the name hosts see when the room joins.
- **Repeat** for Room 2 and Room 3.

  You should now see one active Basic user per room, each named after its room.

> **GOOD TO KNOW**
>
> The room user never hosts anything and needs no licence. If you skip steps 2 to 4, rooms can still join meetings hosted on your own Zoom account; they just cannot join meetings hosted by other companies.

5 Put the credentials on each device

- **Make the file.** Copy `deploy/rooms/zoom-credentials.example.json` to `zoom-credentials.json` and fill in the six values: the step 1 client id and secret (the development pair until step 2 is approved, the production pair after), the step 3 account id, client id and secret, and that room's user as `room_user`. Each room gets its own file because `room_user` differs.
- **Copy it to the device**, for example `scp zoom-credentials.json pi@crystal-meet-room-1.local:~/`.
- **A device you are installing now.** Add `--zoom-credentials ~/zoom-credentials.json` to the installer command from the setup guide.
- **A device that is already installed.** Type `sudo install -o pi -g pi -m 600 ~/zoom-credentials.json /etc/croom/zoom-credentials.json` (your username instead of pi), then `sudo systemctl restart croom`.
- **Check it.** Type `/opt/croom/venv/bin/croom --check-zoom -c /etc/croom/config.yaml`. It reads:

```
Zoom Meeting SDK: app AbC123xYz, signature minted (Zoom checks it only when a meeting is joined)
Zoom account: server-to-server token obtained for account 9XyZ12ab
Zoom room user: room1@crystalpm.com, ZAK obtained (valid 2 hours)
Ready: this room can join meetings on your own Zoom account now; meetings hosted by other
accounts also need Zoom's approval of the SDK app, and the first outside-hosted meeting proves it.
```

- **Delete the copy in your home folder.** Type `rm ~/zoom-credentials.json`.

You should now see the check end with "Ready".

6 Join a test meeting

- **Inside your account.** Have a colleague start a Zoom meeting and paste its invitation link, passcode included, into the room page's **Join with a link**, or book it on the room's calendar and press **Join now**. To join with the meeting id instead, type it in the same box: a **Passcode** field appears under it, so type the meeting's passcode there and press **Join**. A booking whose Zoom link has no passcode in it is joined with the one written in its invitation text ("Passcode: 123456"). Zoom's meeting appears on the TV within about fifteen seconds. This works from day one, with the development pair.
- **Outside your account,** once step 2 is approved and the production pair is on the device. Join a meeting hosted by someone outside the company, or by a personal Zoom account. The room appears in the participant list under its own name, for example Room 1. If the host uses a waiting room, the room page says it is waiting until the host admits it. This step is the proof for the outside-host route, which has not yet been verified against a live meeting: if Zoom refuses with an authorization message even though the app is approved and the check passed, tell whoever set this up; the fallback is fetching the ZAK through the SDK app's own OAuth.
- **Controls.** From the table page press **Mute**, **Turn camera off**, then **Leave** and **Tap again to leave**.

You should now see both meetings join, the room named after itself in the outside one, and the TV return to the room's idle screen after leaving.

IF SOMETHING IS OFF

Troubleshooting

There is no Develop menu in the Marketplace

The signed-in admin's role lacks the **Zoom for developers** permission. The account owner adds it under **User Management › Roles** (Role Settings › Advanced features), or does steps 1 to 3 personally.

The check says "not configured"

It names the value that is missing or still a placeholder in `/etc/croom/zoom-credentials.json`. The four server-to-server values and **room_user** go together: fill in all of them or none.

The check says Zoom refused the credentials

The account id, client id or secret of the step 3 app is wrong, or the app is not activated. Copy them again from App Credentials and check the Activation page.

The check says no such user, or a scope is missing

The **room_user** must be an active user on this Zoom account. "Lacks a scope" quotes the scope Zoom wants; add that scope to the step 3 app under **Scopes** and activate it again.

The room page says the meeting is not authorized

The meeting is hosted by another account. Three things must all be true: the SDK app is approved (step 2), the device holds its production pair, and the file has the server-to-server values and **room_user**. Fix what is missing, restart the service and run the check. If all three are true and Zoom still refuses, the outside-host route needs the fallback described in step 6.

Wrong passcode or meeting not found

Paste the full invitation link with its passcode part, or type the passcode in the **Passcode** field that appears under the link box for a meeting id or a link without one. After a "Couldn't join" that says "Zoom needs the meeting passcode", the card keeps the link and shows the field: type the passcode there and tap **Join again**. Zoom's exact reason shows on the room page.

The room joins muted, or a tap on the table says "try again"

The table shows mute and camera as Zoom has them, and a change the host makes shows within a few seconds. At join the room's camera setting (**camera_default_on** in the room's config) applies both ways: Zoom's video is started or stopped to match it. The microphone setting (**mic_default_on**) applies only towards muted: a room set to join muted is muted, but when the room is set to join with the microphone on and Zoom has it muted (the host's mute on entry, for example), the room stays muted and the table shows **Unmute**, one tap away. A tap that Zoom cannot carry out (the camera still starting, no camera Zoom can use, a host who does not allow participants to unmute) answers in Zoom's words on the meeting screen, and the meeting goes on. To see what the device's page sees (Zoom's field names and its buttons' wording) during a meeting, open **http://<device>:8080/api/meeting/diagnostics** from any browser on the office network and send the result with your report.

The TV stays dark and the page says the SDK did not load

The device could not reach **source.zoom.us**. Check the device's internet access and any web filter, then try again.

GOOD TO KNOW

The room joins meetings as its own Zoom user through Zoom's supported Meeting SDK, so it is never mistaken for a bot. To take that away, deactivate the two apps or the room users in Zoom.